📜 Exemple de Pipeline CI/CD : Déploiement Automatique sur un Serveur de Production

💡 **Objectif :** Ce pipeline GitHub Actions permet **d'envoyer automatiquement** votre application sur un **serveur de production** après chaque **push sur** `main`.

📌 **Ce que fait ce workflow :**
✅ **Construit l'application** (ex: Node.js, Python, etc.)
✅ **Se connecte en SSH au serveur de production**
✅ **Envoie les fichiers sur le serveur** (via `scp`)
✅ **Redémarre le service (Docker, PM2, systemctl, etc.)**

🚀 Pré-requis pour ce workflow

- **Un serveur distant** (VPS, cloud, dédié)
- **Un accès SSH** au serveur (via clé privée)
- **Un dossier où déployer votre application** (`/var/www/app/` par exemple)
- **Un script pour redémarrer l'application** (`pm2 restart app`, `docker-compose up -d`, etc.)
- **Ajout des secrets GitHub** (`PROD_SERVER_IP`, `PROD_SSH_USER`, `PROD_SSH_KEY`)

📜 Fichier `.github/workflows/deploy-to-prod.yml`

```yaml
# 🏷️ Nom du workflow
name: Déploiement Automatique en Production

# 🎯 Déclenchement du workflow sur chaque push sur "main"
on:
  push:
    branches:
      - main
  workflow_dispatch:  # Permet un déclenchement manuel

# 🏗️ Définition des jobs
jobs:
  deploy:
    runs-on: ubuntu-latest  # Utilise un serveur Linux GitHub Actions
    steps:
      - name: 📥 Cloner le dépôt GitHub
        uses: actions/checkout@v4  # Télécharge le code source du projet

      - name: ⚙️ Installer les dépendances
        run: |
          npm install  # Exemple pour un projet Node.js
          npm run build  # Génère les fichiers de production

      - name: 🔐 Configurer la clé SSH
        run: |
          mkdir -p ~/.ssh/
          echo "${{ secrets.PROD_SSH_KEY }}" > ~/.ssh/id_rsa
```

```
          chmod 600 ~/.ssh/id_rsa

      - name: 🚀 Copier les fichiers sur le serveur de production
        run: |
          scp -o StrictHostKeyChecking=no -r ./dist/* ${{ secrets.PROD_SSH_USER
}}@${{ secrets.PROD_SERVER_IP }}:/var/www/app/

      - name: 🔄 Redémarrer l'application sur le serveur
        run: |
          ssh -o StrictHostKeyChecking=no ${{ secrets.PROD_SSH_USER }}@${{
secrets.PROD_SERVER_IP }} << 'EOF'
            cd /var/www/app/
            pm2 restart app  # Commande pour redémarrer une application Node.js
            echo "✅ Application redémarrée avec succès !"
          EOF
```

🛠️ Explication du Workflow

◇ 1️⃣ Construction du projet

✅ **Clone le dépôt GitHub** (`actions/checkout@v4`)
✅ **Installe les dépendances** (`npm install`)
✅ **Construit l'application** (`npm run build`)

◇ 2️⃣ Connexion SSH et Déploiement

✅ **Ajoute la clé SSH dans GitHub Secrets (`PROD_SSH_KEY`)**
✅ **Utilise `scp` pour transférer les fichiers de build vers `/var/www/app/`**
✅ **Se connecte en SSH et redémarre l'application avec `pm2 restart app`**

🔐 Sécurité : Protéger l'Accès au Serveur

➡️ **Ne jamais stocker les identifiants en dur !** Utiliser **GitHub Secrets** :

- **PROD_SERVER_IP** → Adresse IP du serveur
- **PROD_SSH_USER** → Nom d'utilisateur SSH (ex: `ubuntu`, `root`)
- **PROD_SSH_KEY** → Clé privée SSH (`id_rsa`)

➡️ **Ajoutez votre clé publique** (`id_rsa.pub`) dans `~/.ssh/authorized_keys` sur le serveur.